Assignment in the WASP Software Engineering Course Module 2026

Ella J. Schmidtobreick

1 Introduction

My research lies in the intersection of machine learning, linear algebra, and numerical optimization, focusing on learning-based methods for solving linear programs (LPs) and quadratic programs (QPs). The central goal is to improve classical optimization algorithms, such as active-set and interior-point methods, by integrating learned components that exploit problem structure and accelerate convergence. This is motivated by the observation that many traditional solvers rely on hand-crafted rules that may not fully exploit statistical regularities in real-world problem instances, whereas learned models can adapt these decisions from data and improve performance within iterative optimization schemes.

In terms of machine learning methods, I primarily work with graph neural networks (GNNs). Therefore, optimization problems are represented as graphs where variables and constraints correspond to nodes and edges encoding structural dependencies, enabling GNNs to learn structure-aware representations that can support algorithmic components such as constraint selection, search direction prediction, or preconditioning strategies. The prediction is then used as additional information for the solver to accelerate the convergence.

From a software engineering perspective, this research naturally leads to complex AI systems that go beyond isolated models. The training process, data generation, solver integration, and evaluation pipeline must be carefully engineered to ensure correctness, scalability, and reproducibility. This makes principles from software engineering for AI systems highly relevant, especially in terms of modular system design, testing of learned components, and robust integration between machine learning models and numerical solvers.

2 Lecture Principles - Reflection on testing

The part of the lecture regarding quality analysis and testing was very interesting to me due to different aspects. First, I feel like I never properly learned testing and second, there is not one general way on how to test. A very interesting concept that stuck to my mind is the difference between validation and verification, which is distinguished in validating if you're building the right product and verifying if your product works correctly. In my work, this distinction is usually less critical, since my projects have a very clearly defined scope, which is accelerating a specific algorithm. The result can also easily be measured and usually different components in the algorithm can be separated quite well from one another, such that I work on separately defined subtasks. Therefore, I usually focus on verifying that each subtask works as it should. It would be something else, if you have a huge data science project with e.g. health care data and different stakeholders, communicating different goals. In this case, it is not only important to verify that your product works but also to validate if you pursue the right goals.

To stay in the area of testing, the different test levels were also quite interesting to me. Spanning from unit tests for single code units to integration tests for multiple code units combined and system tests for testing the entire system until finally going through the acceptance test, which tests the system from the

perspective of the end-user. In my work, I mostly focus on the first three test stages, and usually do not do acceptance testing, since I am not the end-user and I do not test my accelerated system in any kind of application, where it might be coupled with other algorithms. Furthermore, I sometimes have a hard time breaking my code down into small functions and end up writing too complex functions. Therefore, I want to focus more on doing so in my current and future projects, which would then also refer to unit testing. Usually, I always test from the smallest until towards multiple units used together, which relates to integration tests, to then in the end test the whole system, referring to system testing. One issue, I also generally have in the area of testing, is that there is no universal way of learning how to test properly, such that it is always quite a lot of try and error for me. It becomes especially challenging, if I try to test iterative processes that converge, but the true solver, which acts as baseline comparison, does not take the same intermediate steps, as it might be numerically optimized or uses acceleration methods, such that testing becomes difficult as the intermediate results might not be equal.

3 Guest Lecture Principles

One line that really got stuck in my mind from Per's guest lecture was the definition of Software Engineering. First of all, to me, Software Engineering is always a mysterious term, which feels hard to grasp. When he explained it as "a structured approach to create reliable software", I started rethinking about my understanding of what a software engineer is. I do write code and algorithms in my work, test them and compare result. Am I a software engineer then? Before the course, a software engineer, to me, was always someone providing the structure but not the functionality, but maybe my understanding was a bit too narrow. I'm still not fully certain if any developer is then a software engineer, since this is how the definition sounds to me, or if one needs to have a higher level understanding of what is being built, to call one self a software engineer.

As a specific part of the definition, I stumbled across the keyword "structured". I would say, I am personally structured, but when it comes to writing code and therefore, my touchpoint with software engineering, I would like to be more structured then I am. The way I work feels a bit like creative chaos, which is hard to show to other people and get back into it, such that I would like to be a bit more structured and not just try a lot of different things. In my work, the missing structure also often results from naturally growing methods and experiments, such that I start with something basic, but add more and more functionalities to a certain method instead of writing a new distinct function. Another aspect which contributes to that, is, that while developing, usually I am the only person, who actually looks at the code and works with it, but when the work gets published, the code should also be understandable for others. Therefore, it feels like there is no need to be structured from the beginning on, since I just need to understand my own work, in comparison to maybe a company, where different teams of developers work on the same task. Getting more structured in my software engineering process, while keeping a certain creative chaos, is certainly something I am trying to develop.

Another sentence that stuck to my mind is "Less routine work is not automatically better work". I would argue that a normal human can not work 8h a day with the highest concentration possible, such that it definitely sometimes helps to have standardized or more boring tasks to do, to still stay productive, but also reduce the cognitive intensity. I can definitely feel that in my work. Especially, when I finish a certain part of my work and need to orient myself, what to do next, I often do a simple task in between to relax my mind and therefore be prepared to dive deeper into the next topic. When this work will be automated by AI, I strongly believe that humans still need a break, and can not work continuously be on full cognitive intensity, but also need to recover in between. This rises the question if AI really makes us more productive then or it just feel more productive, since the AI can carry out repetitive work in less time than a human could.

4 Data Scientists, Software Engineers, and AI Engineers - book chapters

In the book [4], data scientists are said to work with data in every aspect, including their statistics and patterns. These are analyzed by machine learning algorithms, such that developing those is a key task of a data scientist. In the book, data scientists are described as purely data and model focused, which I would partly agree with. In my own work as a (data) scientist, I mostly work alone on projects, such that even though my main focus is the model, I also develop the infrastructure and architecture around. Further, there are some other very specific statements given, that I disagree with, which I want to elaborate on in the next paragraph.

To be clear, my reference of a data scientist is mostly myself and colleagues, all working in academia and not in any big tech companies, such that this view is definitely biased. From this perspective, the framing of a data scientist in the book is a bit to narrow. The book claims, that data scientists focus on building models, which I would generally agree with, but then they mention, that data scientists love hyperparameter tuning and feature engineering. I would claim that especially hyperparameter tuning is one of the most annoying parts of the model development, which I generally not really enjoy. It is mentioned, that another main task is the collection and preparation of data, which definitely is an important part for a lot of data scientists working at real-world applications. Since I work more on the theoretical side of data science, I usually just generate data for known problems as basis for my research, such that gathering and cleaning the data is a negligible part of my work. Further, the book claims that data scientists often work in jupyter notebooks. This is for sure an important tool for the exploration phase but for everything that goes beyond that, jupyter notebooks are not sufficient. In general, I would claim that the major task of being a data scientist is to understand the data one is working with and to exploit their statistics. This summarizes this job for me a bit better than the description in the book.

One thing I was majorly irritated by, was the comparison between the educational backgrounds of data scientists and software engineers (and especially that this was mentioned as the first aspect in the descriptions). The book claims, that data scientists tend to have a higher educational and academic training then software engineers, which in my eyes makes the reader biased from the beginning on. If this claim is actually true, is very hard to verify for me, since I am in an academic environment as a data scientist and therefore the claim holds for me and my colleagues. Since I work in an academic bubble, it is also very hard to falsify the claim.

On the other handside, the book states that software engineers have a more highlevel overview, where a machine learning model is only one component in a huge system. It is claimed that they are not as research oriented as data scientists, but focus more on delivering products for customers. This is very hard for me to actually verify, since I am not working in a software engineering nor economic environment. It is further stated that software engineers constantly apply engineering judgment to navigate trade-offs between different factors, such as uncertainty and budget constraints. I would claim, that data scientists need to deal with this equivalently, when judging if more time and computational resources (and therefore money) can be used to improve the models quality. Further, the book mentions, that software engineers built the whole architecture, where machine learning just contributes as one component. I would say, that this is true, but nevertheless a data scientist also needs to built a ML architecture, including a whole pipeline, which could also be described as AI Engineering. However, this is, especially in research, less extensive then building a customer ready product.

Concluding, I would say, that for a successful real-world product with a machine learning component, it is necessary for data scientists and software engineers to efficiently work hand in hand. Given my current experience, nowadays data scientists and software engineers are not as distinct roles as they might have been in earlier times and will probably also converge and merge even more. This effect is not only necessary but also supported by agentic AI systems and coding assistents, which makes it easier and especially more accessible to set up or implement something in a new language or a new field of computer science. The book also clearly states, that interdisciplinary teams are needed for a

successful end product, such that different expertise are brought together. For that the communication and appreciation between different team members is key.

5 Paper Analysis

5.1 Paper 1 (CAIN conference paper)

As the first paper [5], I picked "An Exploratory Study of V-Model in Building ML-Enabled Software: A Systems Engineering Perspective", since the V-model is a very common model, and was also mentioned in the lectures of this course.

The paper addresses the interdisciplinary communication challenges between software engineers and data scientists. One issue is, that tradition software development life cycles do not fit well with the new way on how machine learning models are developed, explored and build. Therefore, the current processes have to be adapted, such that an interdisciplinary collaboration works smoothly for everyone. Specifically, the V-model is discussed here, where "V" stands for validation and verification as well as the shape of the model. The paper is based on 11 interviews with different software engineers and data scientists, who all had experience with machine learning based projects. Based on these interviews, the authors derive several propositions on how the V-model can be adapted for ML-enabled software. The paper concludes that following the V-model strictly for an ML-enabled application is not fully functional and requires additional adaptations that directly address the challenges arising during collaboration.

One challenge is that the impact of ML techniques is often difficult to measure from the customer's perspective. It is hard to demonstrate precise improvements in business metrics after integrating an ML component. Another issue in the communication is the still limited knowledge about AI and ML, which often leads to unrealistic expectations regarding the capabilities of the systems. Therefore, applying established software engineering principles, such as explicitly defining requirements and corresponding validation activities, can help to create more realistic expectations among all stakeholders.

Another important aspect discussed in the paper is that ML engineers should already be involved during the requirements engineering phase instead of only during implementation. Since the available data and the limitations of ML models influence what is technically feasible, these aspects should already be considered when defining the system requirements.

One of the key improvements highlighted by the paper is that using the V-model promotes a system-wide perspective, which defines clear responsibilities and system boundaries. This holistic view is often missing when ML-enabled products are developed, making it more difficult to separate software-related issues from data- or model-related problems. Furthermore, the paper emphasizes that validation should not only focus on whether the software implementation is correct, but also on whether the ML-enabled system actually solves the intended problem and satisfies the needs of its users.

This paper definitely relates to my own research, since I develop only a small ML component that could later potentially be integrated into a bigger project. However, while developing my component to accelerate algorithms, I do not have a particular use case in mind, where the model should later be applied to, such that an early discussion about integration and its requirements is infeasible. A potential application of my research could be an electric grid and the optimization problems, that are needed to be solved there.

The paper is in some sense a reminder, to not only think about prediction acurracy and computational speed, which hopefully results in a better model, but also the later integrability. It could be an idea to validte my models in a complete software system, including robustness, reliability, reproducibility, and the interaction with the underlying optimization algorithm. In addition, defining validation criteria already during the design phase instead of only evaluating the final model would better align the research with the engineering principles discussed in the paper and support the integration of ML-based

optimization methods into larger AI-enabled software systems.

5.2 Recent paper related to technical debt in AI systems

As second paper, the paper "Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development" [2] was chosen. Since more and more AI-generated code is used in a project, knowing how to deal with it and what technical debts may result from it is more important than ever. This is true for general projects, but also my own research, such that I can personally be aware of potential risks and adapt my personal workflow accordingly.

The paper is a multivocal literature review, which is therefore not only based on academic publications, but also on, what they call "grey literature", referring to industry reports, blogs and online forums. After reading the paper, I would say, that most debts are mentioned in both sources, while just a few debts were only mentioned in one of the categories.

After describing how they curated the 104 sources, they answer 5 research questions. To me the first one was most interesting, but I will briefly mention all of them in the following paragraph.

Question 1 discusses **what forms of technical debt are introduced by integrating LLM-generated code into software projects** [2]. The analysis is based on the 13 traditional debts mentioned in [1], and 6 additional debt types were defined. In both, the grey literature as well as formal literature, two major categories of debt were identified: code debt and design debt. Code debt includes the issue, that the logic and structure of AI provided code is not fully understood by the responsible developers, such that duplicates, inefficient code, and increased code complexity is common. Design debt refers in this context especially to how the code is design and that the model does not understand your code structure, such that the generated code does not necessarily align with your existing code base. It is mentioned, that the big issue in both cases, is that a huge amount of code can be generated in a short amount of time, which increases the risk of not having a proper overview as a human.

In the grey literature especially governance debt is also highlighted, which refers to oversights, that create long-term burdens. Further debts as ethical debts, prompt debts, and ownership debts are mentioned, even if less often. The academic literature also mentions data debt as a risk, which relates data-related conditions that influence AI-generated output, which can then mean higher maintenance afterwards. This chapter once more reminded me, to not let generative AI write my code blindly and how important it is to double check the outcome and changes made by models to always be fully aware of how each function works.

Research question 2 refers to **strategies or guidelines to mitigate technical debt introduced by LLMs**. Both types of literature mention prompt engineering as a useful tool as well as additional tooling and human-in-the-loop frameworks. It is mentioned, that it is essential to still have a human overseeing and verifying the output, such that code is not written blindly into the project. A very interesting idea mentioned in the paper is to treat the LLM-generated code similar to code generated by a junior developer, such that one is aware that the code requires reviewing and refinements. Additionally, the formal literature mentions data quality, which emphasizes preprocessing the data, which can reduce further code issues when generating code.

Similar to the last chapter, this chapter made me further aware of how important I am in the whole coding process, especially if I use generative AI and it seems like I am not needed anymore to actually write the code. Therefore, one should more treat generated code as a suggestion, but not follow if blindly. That already starts with how to name variables, to be sure what is meant. To me personally also commenting the code is a big aspect here, since if I do not have written the code myself documenting it properly forces me to understand what is happening.

Research question 3 deals with the **tools and techniques to help practitioners detect and measure technical debt in LLM-generated code**. The paper claims, that there are not many tools available

yet, one that made me very curious is SonarQube, which is a general-pupose code quality analysis tool. Since I had never heard of these tools, but feel like my code quality could be improved, I integrated it into my research workflow right away, such that I can test it on my current projects now, and hopefully gain a higher code quality in the future.

Question 4 refers to if **there exist benchmarks or datasets to evaluate the technical debt or quality of LLM-generated code**. For both types of literature, the answer to this question is that no general benchmarks exist.

The last question discusses **metrics that capture the debt introduced by LLMs**. Across both literature types, it was identified that there is a lack of concrete metrics, that are adopted. There are a few ideas like Data Smell Strength and Data Smell density [3].

As mentioned above, I already adapted a code quality tool into my personal workflow after reading this paper, such that I can hopefully produce higher quality code. The paper also made me especially aware how important it is to actually write the code yourself. Not only is it you being responsible for the outcome, but you should also be able to follow and explain every single step, which is especially important in AI research, as the model is a black box already. To be aware of if the outcome makes sense, it is important to know all the steps and concepts used in detail and not rely on an AI, even though it might be easier and faster, as it uses less mental resources. As a result, if I use AI-generated code, I want to either use it as inspiration and write a similar function myself or only let very specific and small chunks of code be written by an LLM, which is easier to verify the correctness of the generated cod.

6 Evidence, originality, and GenAI transparency

All the sources used, can be found in the reference section at the end of this report.

Chapter 2 and 3 are based on my own notes of visiting the course in Gothenburg as well as the (guest) lecture notes published at the Canvas page relating to this course. Chapter 4 is fully based on a book [4], as stated in the instructions. Chapter 5.1 is fully based on the peer-reviewed and published paper [5]. Chapter 5.2 is based on the just accepeted and therefore peer-reviewed paper [2].

Generative AI was used to find the two papers in Section 5.1 and 5.2, but for no other step when writing the report.

References

[1] Nicolli S. R. Alves, Leilane F. Ribeiro, Vivyane Caires, Thiago S. Mendes, and Rodrigo O. Spínola. Towards an ontology of terms on technical debt. In *Proceedings of the 2014 Sixth International Workshop on Managing Technical Debt*, MTD '14, page 1–7, USA, 2014. IEEE Computer Society.

[2] Ramtin Ehsani, Shriya Rawal, Yuanfang Cai, and Preetha Chatterjee. Faster code, deeper debt? a multivocal literature review on technical debt and its early signs in llm-assisted software development. *ACM Trans. Softw. Eng. Methodol.*, June 2026.

[3] Harald Foidl, Michael Felderer, and Rudolf Ramler. Data smells: categories, causes and consequences, and detection of suspicious data in ai-based systems. In *Proceedings of the 1st International Conference on AI Engineering: Software Engineering for AI*, CAIN '22, page 229–239, New York, NY, USA, 2022. Association for Computing Machinery.

[4] Christian Kästner. *Machine Learning in Production: From Models to Products*. MIT Press, Cambridge, MA, USA, April 2025.

[5] Jie JW Wu. An Exploratory Study of V-Model in Building ML-Enabled Software: A Systems Engineering Perspective. In *Proceedings of the IEEE/ACM 3rd International Conference on AI*

Engineering - Software Engineering for AI, CAIN '24, pages 30–40. Association for Computing Machinery.